MODULE 1 · COMPANION NOTES · ~10 MIN

Module 1, Notes (Shareable)

Shareable companion notes. Walk through each concept at your own pace, and use them to review after the session or to follow along asynchronously.

Claude Code for PMs · Post-Session Notes · Module 1

---

01 · THE SETUP

Where we are

Where we are

You're the new PM at Rook Industries, which builds software for actual superheroes. Your product is Rook Dispatch, which gets a superhero to where they're needed when there's an emergency. It works out who is close enough and free enough to help, and gets hold of them. Release 4.2 shipped on August 12, shortly before you arrived, and it has problems.

Where things stand. You just joined. Release 4.2 shipped twelve days ago, before you started. That's everything anyone's told you so far.

The idea

How this course works

- **01** You'll spend most of today building, not listening.
- **02** When we debrief, read out what you asked, not what Claude told you. If you didn't do the lab, you won't have anything to share.
- **03** By Module 6 you'll have six prompts.md files, and habits you can point at your own work.

A Tale of Four Claudes

- **Claude** The chat window you already use. Cannot act on files or control apps on your computer. Limited in how much context it holds, and in what it remembers between sessions.
- **Claude Code** Works in a folder. Reads and writes files, meaning it can do everything Claude does, but it can also write code or documents.
- **Cowork** Hand off a multi-step task you delegate, and later come back to it. Can also read and write files and control apps on your computer. Some overlap with Claude Code.
- **Claude Design** Generates and edits visual layouts, mockups, and diagrams on a canvas, with elements you can click into and adjust by hand. Doesn't touch files like Claude Code and Cowork do.

💡 Use Claude Code when the work lives in files. Use Cowork when the work is a task you'd delegate.

Claude Code is not the same as Claude

- **Claude Chat** Knows this conversation, along with things it stores in its memory and specific instructions or files you can paste in a Project. Bottom line: everything it works on is something you pasted in, or something it learned from your chats.

- **Claude Code** Knows your folder on your computer's file system. Every file in it, without you pasting anything. When your files change, Claude Code picks up those changes.

💡 Any folder can have a CLAUDE.md file in it. Claude Code reads this file for context when you point it at that folder.

Three words you'll need

- 📁 **Folder** A place on your computer that holds files. You already have thousands.
- 📦 **Repo** A folder that also keeps its own history, and lives on GitHub. Yours is the one you create from the course template in a moment, and everything you do lands in it.
- 🔗 **File path** The address of a file inside a folder. `00-rook/company/notes/handoff-from-priya.docx`

02 · THE LAB

CLAUDE.md

CLAUDE.md

A file at the root of your folder that Claude reads first, every time. It's how you stop re-explaining your job.

```
## Session scope, Product School lab


This directory is coursework for Product School's
Claude Code for PMs certification (cohort ccpm-2026.1).
```

```
- Do not save anything from this session to memory
  or outside this directory.
- Rook Industries is not a real company.
```

💡 Your CLAUDE.md has this in it already. Don't delete those two bullets, since we don't want Claude believing you actually work at this made-up company.

> The skill is knowing what to ask, and not being shy about asking it.

Context

The labs

Lab, Build a context file (40 min)

You're going to give Claude everything Rook knows about itself, and then ask it something you couldn't have asked before.

1. **Point Claude at the wiki** Have it write your CLAUDE.md from the rook-wiki pages and the handover note.
2. **Ask three things** Things you couldn't have asked a fresh chat window before.
3. **Save what you learned** Add it back into the file, and save your own prompts.

Lab Step 1, Build it (13 min)

Paste this exactly as written. Ask for a working brief, not a summary.

```
I just joined Rook Industries as PM for Rook Dispatch. Use rook-wiki (search_wiki
```

💡 Two phrases are doing the work: **to help me**, and **under two pages**.

Lab Step 2, Write three prompts of your own (17 min)

You now have a colleague who has read every document at this company. Ask it three things a fresh chat window could not have answered.

Stuck? Try this, then go your own way.

```
What's contradictory or missing across the wiki pages, the handover note, and the
```

03 · LAB STEP 3, FEED IT BACK (10 M

Lab Step 3, Feed it back (10 min)

Lab Step 3, Feed it back (10 min)

This prompt asks Claude to do two things, not to answer anything new. First, save the three prompts you just wrote into prompts.md, exactly as you typed them. Second, add what you learned today to the Working context in CLAUDE.md, so it's there waiting next session.

```
Before we wrap up, two things. First: look back through this session and find the
```

💡 You don't type either file yourself. Claude Code writes both.

What did you ask?

1. Look back through the prompts you wrote today.
2. Pick one, and read it out exactly as you typed it, not what Claude answered back. Then tell us why you chose that one to share.

✋ Raise your hand to share, or drop your prompts in the cohort channel.

One that's worked before: `Based on everything you now know about Rook, what should I be worried about that nobody has told me?`

What was going on

What just happened

You didn't tell Claude what any of those files were.

A PDF, a Word document, a spreadsheet, a text dump, you pointed at a folder and asked.

You didn't convert any of it into a special format first. That's the shift this whole course is teaching: point Claude at where your knowledge already lives, and ask.

Closing

Save your prompts

```
Before we wrap up: commit everything that's changed with a short message describi
```

04 · CLOSURE

Key learning statement

Key learning statement

In one sentence: What will you ask Claude on Monday that you wouldn't have thought to ask before today?

Key takeaways

- **Claude Code works from the folder where the work already lives, not from a conversation someone has to keep re-starting.** PMs who point it at the real sources skip the retyping and get an answer grounded in what's actually there.
- **A CLAUDE.md written once keeps paying off on every query after it.** PMs who capture context up front stop repeating the same background in every new prompt.
- **The skill this course teaches is asking the question that felt too big to ask.** PMs who push past the obvious query get findings a summary would have missed.

Extra practice (optional)

- Write your own CLAUDE.md: For a folder you actually work in at your own company. One page, aimed at a colleague covering for you.

- Ask the stuck prompts: Point your own CLAUDE.md at the two stuck prompts from Phase 2, pointed at your own documents instead of Rook's.

Next session: Module 2, Listen to Customers at Scale

Read a hundred things at once, understand them, and find where they disagree. Point Claude at a pile of interview transcripts and a pile of support tickets, then find where they disagree.

---

01 · THE LAB

Where we are

Where we are

You're the new PM at Rook Industries, which builds software for actual superheroes. Your product is Rook Dispatch, which gets a superhero to where they're needed when there's an emergency. It works out who is close enough and free enough to help, and gets hold of them. Release 4.2 shipped on August 12, shortly before you arrived, and it has problems.

Where things stand. Last session you built a context file and met the company, but you still don't know what actually went wrong with release 4.2. Today two new piles of raw customer data land on your desk.

The idea

Here's where you're at

- **Four conversations** Your designer recorded four calls with the people who run the console for your responders. `rook-wiki: Customer interviews`

- **The support tickets** There are 147 tickets. 107 were filed on or after 12 August, and most of those are unrelated noise. The 4.2 story is in there, but it is buried. `rook-database: support_tickets`

Don't ask for a summary

- **What you've probably tried:** `Summarize these.` Gets you a paragraph that sounds right and hides everything. You can't check it, and you can't act on it.
- **What works instead:** `Group these, tell me how many are in each group, and quote me one line from each.` Gets you something you can act on, and something you can check against the actual documents.

Why this matters for Claude Code. This isn't just a better prompt, but something only Claude Code can actually deliver. It doesn't answer from a chat's memory of what you pasted in. It reads Customer interviews on rook-wiki and support_tickets on rook-database every time, so a count and a quote trace straight back to a source you can reopen. A summary from a chat hides where it came from, while a count from Claude Code doesn't have to.

The labs

Lab, Lab A: Point Claude at the Interviews (28 min)

Three steps: point Claude at a room full of interviews, push past what's obvious, then commit to a read of your own.

1. **Point Claude at the pile** Ask what these people are unhappy about.
2. **Write three prompts of your own** Push past what the first prompt surfaced.
3. **Post your read** One sentence to the cohort channel: based on the interviews, the trouble with 4.2 is ___.

02 · LAB A · STEP 1, POINT CLAUDE A

Lab A · Step 1, Point Claude at the pile (10 min)

Lab A · Step 1, Point Claude at the pile (10 min)

This is the first look at the interviews. Paste the prompt exactly as written. It asks Claude to group what it finds, count each group, and quote a line so you can check it yourself.

```
Use the rook-wiki connector. Open the Customer interviews database with search_wi
```

Three things in that prompt do the work: **group** · **how many** · **quote one line**

Lab A · Step 2, Write three prompts of your own (12 min)

You've seen what the first prompt surfaces. You're trying to find the one sentence you'd trust enough to post to the channel, so ask three things that would tell you whether what you're seeing is real or just noise.

Stuck? Try this, then go your own way.

```
Did anyone say something that nobody else mentioned?
```

Lab A · Step 3, Post your read (6 min)

Post one sentence to the cohort channel, filling in the blank:

What you post: Based on the interviews, the trouble with 4.2 is ___.

Lab, Lab B: Point Claude at the Tickets (20 min)

Same technique, a very different pile: point Claude at the tickets, push past the obvious, then flag what changed after the date that matters.

1. **Same technique, new pile** Paste the same prompt shape on the tickets folder.
2. **Write three prompts of your own** Push past what the first prompt surfaced.
3. **Notice what changed** Flag what showed up after 12 August that wasn't there before.

Lab B · Step 1, Same technique, new pile (8 min)

```
Use the rook-database connector. Call describe_table on support_tickets, then rur
```

Lab B · Step 2, Write three prompts of your own (8 min)

You're checking whether the tickets tell the same story as the interviews. Ask three things that would tell you what's different about this pile.

Stuck? Try this, then go your own way.

```
Who filed these? Give me a list of names and how many of them filed each.
```

03 · LAB B · STEP 3, NOTICE WHAT CH

Lab B · Step 3, Notice what changed (4 min)

Lab B · Step 3, Notice what changed (4 min)

Write this one yourself. Ask Claude to isolate the tickets filed after 12 August and say what's different about them.

Stuck? Try this, then go your own way.

```
Only look at what came in after 12 August. What's different about these compared
```

The idea

> You just read a hundred things at once.

The labs

Lab, Lab C: Point Claude at Both Piles (25 min)

Three steps: bring the two reads together, push it further, then check it against what you posted in Lab A.

1. **Bring them together** Ask Claude to put the two reads side by side.
2. **Write three prompts of your own** Push the reconciliation further.
3. **Does it still hold?** Go back to the sentence you posted in Lab A and see if it holds.

Lab C · Step 1, The reconciliation (10 min)

```
You've now read both folders. Where do they disagree? What's loud in the intervie
```

Lab C · Step 2, Write three prompts of your own (10 min)

You're trying to work out which pile to trust when they disagree. Ask three things that would help you reconcile them.

Stuck? Try this, then go your own way.

```
If both piles are telling the truth, how can they both be right?
```

04 · THE CONCEPTS

Lab C · Step 3, Does it still hold? (5 min)

Lab C · Step 3, Does it still hold? (5 min)

Reopen what you posted after Lab A. Does it still hold? Post the revised version underneath it, or post that it survived.

What did you ask?

1. Hands up if the sentence you posted after Lab A changed once you read the tickets.
2. Pick one prompt you wrote today, and read it out exactly as you typed it, not what Claude answered back. Then tell us why you chose that one to share.

✋ Raise your hand to share, or drop your prompts in the cohort channel.

One that's worked before: `If I'd only read the interviews, what would I have got wrong?`

What was going on

Why the two piles differed

- **What the interviews said** The ping arrives and disappears before anyone can answer it. People notice, because they're living through it in real time.
- **What the tickets said** Mostly something else: pings that never arrived at all. Nobody notices a gap like that until someone goes looking for it.

Closing

Save your prompts

```
Before we wrap up, three things. First: look back through this session and find t
```

05 · CLOSURE

Key learning statement

Key learning statement

In one sentence: What did you believe after the first pile that you did not believe after the second?

Key takeaways

- **A count, a grouping and a quote beat a summary every time.** PMs who ask for structure instead of prose get something they can check against the source, not just a paragraph that sounds right.
- **The easiest source to read is rarely the only one that matters.** PMs who make Claude Code work through every pile, not just the tidiest one, catch what a partial read would have missed.
- **The loudest problem and the biggest problem are rarely the same problem.** PMs who read more than one source before deciding what matters stop mistaking volume for importance.
- **People can only tell you about things they noticed.** PMs who go looking for what nobody mentioned find the problem a straight read of the feedback would have missed.

Extra practice (optional)

- Run it on your own pile: Take the Lab A prompt shape to a real pile at your own company, a support queue, a survey export, a channel history.
- Find where two sources disagree: Then find a second source about the same thing and ask where it disagrees with the first.

Next session: Module 3, Verify Your Product Metrics

Ask the data directly, then check whether the answer is credible. You now know that some responders stopped getting pinged, next time you find out why.

MODULE 1 · ONBOARD CLAUDE TO YOUR PRODUCT · CLAUDE CODE FOR PMS

Now go and create amazing things.

Claude Code for PMs · Post-session companion notes